

TrustLens Viva Prep

Day 2 — Modules in Bulk, Model Training Story,
Research References, and the Full Code Map

How to use this file

This covers only the modules that are actually built and wired into the live app: Data Ingestion, Fake-Account Detection (both methods), Engagement Analyzer, Credential Extractor, Misinformation Classifier, Trust Score Engine, Authentication, Browser Extension, and the Admin Console. Nothing not-yet-built is mentioned as if it exists — if the panel asks about Deepfake, Visual Analytics, TikTok, or the video transcriber, the honest one-line answer for each is in the "If asked about something not built" box near the end.

Read this first, calmly

You are not behind. Every module below is real, tested, and you already have the numbers memorised from Day 1. Today just adds: exactly where the training file lives, why the fake-follower accuracy number changed, whether any research paper backs each module, and a plain map of which file does what — so if the panel opens your code, you already know what they're looking at.

1. The Fake-Account Model: Where It Lives, 99% vs 95.8%

This is the question you are most scared of, so it goes first. Here is exactly what to say, in order.

Where is the training file?

`backend/train_fake_follower.py`

The one and only script that trains this model. Run it with `python train_fake_follower.py` from inside `backend/` and it retrains from scratch, using data already in the repo.

`backend/archive/fake_follower_detection_algorithms/goyal-dataset/`

The labelled dataset it trains on — 696 real Instagram accounts, each marked genuine or fake by a human.

`backend/fake_follower_model.pkl`

The saved, already-trained model. This is what the live app actually loads on every scan — you do not need to retrain it to demo the project.

What actually happened (say this plainly, it makes you look more credible, not less):

- Early in the project, a model was trained in a Jupyter notebook and reported 99.3% accuracy. That number was saved and reused everywhere — website, slides, memory.
- While auditing every scoring module before the panel, I tested that saved model directly against 696 known accounts. It predicted **"genuine" for every single one** — including the ones that were actually fake. That is exactly 50% accuracy on a balanced set: pure chance, dressed up as 99.3%.
- The cause: the notebook scaled the numbers before training (`StandardScaler`) but only saved the trained model, never the scaling step. So every time the live app fed it a real follower count in the millions, the model was seeing a number completely outside the range it was trained on, and it always fell down the same branch.
- Fix: `train_fake_follower.py` retrains the same algorithm (Random Forest) the same way, but on **raw, unscaled numbers** — a Random Forest never needed scaling in the first place, it just splits on thresholds, so this removes the entire class of bug permanently and can never happen again.
- The honest result on data held back from training: **95.8% accuracy**.

The one line to say if the panel asks "why did the number change"

"The first version scored 99.3% but that number was never trustworthy — it turned out the model was broken and just guessing 'genuine' every time, which happens to score high on a mostly-genuine test set. I found this myself while auditing the project, fixed the actual bug, retrained it properly, and can now reproduce 95.8% on data the model never saw during training. It's a lower number, but it's the first one I can actually stand behind."

This is genuinely a strong answer in a viva — it shows you test your own work instead of trusting a number because it looked good. Do not be scared of it.

2. Research References — Only Where Real

Your supervisor wants you to say "sir, we used this method from this paper" where it is genuinely true. Below is exactly which modules have a real published reference behind their core technique, and — just as important — which ones honestly do not, so you never get caught citing something that isn't there.

Modules with a real research paper behind the technique

Module	Technique	Paper you can cite
Misinformation Classifier	Fine-tuned XLM-RoBERTa (transformer language model)	Conneau et al., "Unsupervised Cross-lingual Representation Learning at Scale", ACL 2020. This is the paper that introduced XLM-RoBERTa itself.
Fake Follower Detection (graph half)	Louvain community detection, used to find bot clusters	Blondel, Guillaume, Lambiotte, Lefebvre, "Fast unfolding of communities in large networks", Journal of Statistical Mechanics, 2008. The standard paper for the Louvain method.
Fake Follower Detection (live model)	Random Forest classifier	Breiman, "Random Forests", Machine Learning, 2001. The original paper for the algorithm itself (not for this specific dataset).

How to phrase this to the panel

"We didn't invent the Random Forest or XLM-RoBERTa — those are established, published methods. What we built is the pipeline around them: which features go in, how the training data was assembled, and how the output feeds into our own Trust Score formula. The Manhattan-distance-plus-Louvain clustering follows the exact method our scope document specifies, and Louvain itself is a widely used, published graph-theory algorithm from 2008."

Modules with NO academic paper behind them — say this honestly

Engagement Analyzer — be upfront about this one

There is **no research paper** behind the engagement-rate benchmark numbers (5% for small accounts down to 0.8% for huge ones). These are commonly quoted social-media-marketing industry benchmarks, the same kind of numbers agencies and marketing blogs use — not a peer-reviewed academic figure. If asked directly: "This module uses industry-standard engagement benchmarks rather than a single academic paper — the scope document only asks for basic rule-based logic here, not machine learning, so this is intentional, not a gap." Do NOT claim a fake citation for this one — it is much safer to say plainly that it is a designed heuristic than to be caught with a made-up reference.

Credential Extractor — also rule-based by design

Pattern-matching against a list of known credential abbreviations (MBBS, MD, PhD, CFA, etc.). This is regular-expression logic you designed, not a trained model and not from a paper. The scope document itself only asks for extraction + a confidence score here, which is what was built.

3. How Engagement Is Actually Calculated

Explain it in this order — it maps exactly to what the code does, top to bottom.

- **Step 1 — Engagement rate.** Average (likes + comments) on recent posts, divided by follower count, as a percentage. This is the standard industry definition of engagement rate.
- **Step 2 — Pick a benchmark by account size.** Nano (under 10K followers) expects 5%, Micro (10K-100K) expects 3%, Macro (100K-1M) expects 1.5%, Mega (over 1M) expects 0.8%. Bigger accounts naturally get a lower percentage of their huge audience actually engaging — this is well known in social media marketing.
- **Step 3 — Ratio and score.** Divide the account's real rate by its benchmark. A ratio of 1.0 means "exactly typical for its size". This ratio is then converted to a 0-100 score using a smooth curve, not a hard cutoff.
- **Why a smooth curve and not fixed steps:** the first version had 4 fixed bands (20/50/75/90), so an account at 0.74% of its benchmark scored 20 and at 0.75% — a 0.01% difference — scored 50. That is an 8-point jump in the final Trust Score from a difference too small for a human to even perceive. Found this during the audit and replaced it with a smooth mathematical curve so a tiny change in engagement only ever produces a tiny change in score.
- **Step 4 — What if Instagram returns zero posts?** This really happened once in a saved scan — a network hiccup meant no posts came back at all, which used to be silently treated as "0 likes" and wrongly scored as highly suspicious. Now it is reported as "Not measured" and left out of the Trust Score entirely, the same honest handling used when there's no caption text to check for misinformation.

backend/engagement_analyzer.py

analyze_engagement() — everything above, in about 60 lines.
analyze_comment_authenticity() — a separate check on whether comments look like real conversation or repeated bot text.

4. Bulk Q&A; — By Module

Read every question here even if it feels repetitive — the panel often asks the same thing three different ways to see if your answer changes. It shouldn't.

Data Ingestion

Q: Where does your data come from?

A third-party service called RapidAPI, specifically an 'Instagram Scraper' API. We send it a username and it returns the public profile info, recent posts, and comments — the same data you'd see just visiting the profile yourself.

Q: Why not Instagram's own official API?

Instagram's official Graph API requires a Meta business review process that takes weeks and needs a registered business — not realistic for a student project timeline. RapidAPI gives the same public data without that barrier.

Q: What happens if the API is slow or fails?

Every call has an explicit timeout (20 seconds for profile and posts, 15 for comments) so a scan can never hang forever — worst case it fails cleanly within under a minute and shows an error instead of freezing.

Q: Can you scan a private account?

No — only public accounts, by design and by what the API can even return.

Fake-Account Detection (both methods)

Q: What features does the live model look at?

Follower count, following count, post count, whether it has a profile picture, bio length, whether it has an external link, whether it's private, plus four username/name features: how many digits are in the username, how many words are in the full name, digits in the full name, and whether the name equals the username.

Q: Does it look at the account's followers?

No — that's the one thing people always assume it does. This model only looks at the account's OWN profile. That's exactly why the site calls it 'Account Authenticity' and 'Fake-account likelihood', not 'follower analysis'.

Q: So how do you check the followers themselves?

That's the second, separate method — the graph-theory clustering. It groups accounts by how similar they are to each other and flags dense clusters of near-identical accounts as likely bot farms.

Q: Why do you need two different fake-account methods?

They answer two different questions. The live Random Forest asks 'does this one account look fake?' on every single scan, instantly. The graph method asks 'inside this pool of accounts, which sub-groups behave so identically they were probably mass-produced together?' — that needs a whole pool of accounts to compare, not just one.

Q: Why isn't the graph method running live on every scan?

Instagram's follower-list API endpoint reliably fails for everyone, confirmed by testing it repeatedly in RapidAPI's own console — it's a provider-side block, not our bug. So this method runs offline on a labelled research dataset of 1,890 accounts instead. The code is written so if that endpoint ever starts working, the exact same clustering function runs on live data with no rewrite needed.

Q: What's the accuracy of the graph method, and why is recall so low?

94.7% precision, 38.9% recall, 81.6% accuracy. Low recall is inherent to clustering, not a flaw: it can only catch bots that arrive in identical batches. A single lone fake account with no near-identical twin has nothing to cluster with, so it's invisible to this method — which is exactly why we keep the live per-account model as well.

Q: What's a known weakness of the live model?

It can score a genuinely new, small, real account as bot-like, because the 'genuine' examples it trained on are mostly established accounts, not brand-new ones. Say this plainly if asked — it's a real, honest limitation.

Misinformation Classifier

Q: What model is this and what does it do?

A fine-tuned XLM-RoBERTa transformer model, trained to sort text into 6 classes: credible, health misinformation, political propaganda, financial scam, sensational clickbait, and Urdu misinformation.

Q: What's the accuracy and is it a real held-out test?

77% accuracy, measured on a genuine held-out test set the model never saw during training or validation — 4,640 examples. Not a number from training data.

Q: Why is it a hybrid and not pure AI?

Because testing found the AI model alone was unreliable on one specific class — it flagged ordinary promotional captions ('link in bio', '10% off with code X') as financial scams, while sometimes missing real scams phrased casually. So a 'financial scam' label from the AI now only counts if a hand-written rule also confirms a real red flag (like a guaranteed-return promise or a disguised link). Every other class still comes straight from the AI model, unmodified.

Q: Does it check Urdu text?

Yes — it has its own dedicated 'urdu_misinformation' class, trained on the UrduFake dataset, kept separate from the English political class because merging them hurt accuracy in testing.

Q: What does it actually check on a scan?

The bio, plus every fetched post caption individually. Whichever single text scores the highest risk is what feeds the Trust Score — one bad caption is enough to matter even if the bio is clean.

Q: What's the honestly-documented weak spot?

Health misinformation was trained mostly on formal COVID-era claims, so informal wellness-scam wording ('this root cures diabetes') sometimes gets the wrong sub-category — but it still correctly flags it as misinformation overall, just not always in the exact right bucket.

Credential Extractor

Q: Is this AI?

No — pure pattern matching (regular expressions) against a list of known credential abbreviations and titles. No model, no training. Say this clearly if asked; do not call it AI.

Q: Does it verify the claim is real?

No, and this is an honest, documented limitation. It only detects that a claim was made and combines that with Instagram's own verification badge status into a confidence score — it does not check any external registry or LinkedIn.

Q: Why not verify externally?

We tried — Google's search API required linking a real billing account even for free-tier usage, which was too risky to keep active this close to the panel. Deliberately deferred, not forgotten.

Trust Score Engine

Q: How is the final score calculated?

Four modules can each contribute: Fake-Account (base weight 25), Engagement (20), Misinformation (25), Credential (10). Whichever of these actually applied to this specific scan have their weights scaled up to sum to exactly 100% — nothing is ever left as a guessed or placeholder number.

Q: Give a concrete example.

If a bio has no credential claim and there's no caption/bio text to classify, only Fake-Account and Engagement apply, and their 25/20 becomes 55.56%/44.44% — still summing to exactly 100%, just re-scaled.

Q: Why not just use a flat default score for a module that doesn't apply?

We used to — a flat neutral 70. That's dishonest for the same reason a placeholder Deepfake score would have been: it's a guessed number presented as real signal. Removed entirely in favour of re-normalizing across only the modules that actually ran.

Q: What's the kill switch?

If the bot percentage from the fake-account model is 85% or higher, the score is capped at 15 regardless of anything else — an account that's almost certainly a bot farm shouldn't be rescued by good engagement numbers.

Q: Is the score deterministic — same input, same output?

Yes, verified directly: the same account scanned three separate times produced the exact same score each time. Nothing here is random.

Authentication

Q: Is this 2FA?

No — say this precisely if asked. It's one-time email verification at signup. After that, login is password-only, forever. A genuine 2FA-at-login version was actually built first and then deliberately replaced because it made no sense for this kind of app.

Q: How are passwords stored?

Hashed with bcrypt — never stored in plain text, and the hash can't be reversed back into the password.

Q: What stops someone signing up with a fake email?

The verification code is required before the account can ever log in. A fake address never receives the code, so that account is permanently unusable — verified by testing a real fake address end-to-end.

Browser Extension

Q: Who built this and does it use the same models as the website?

Built by teammate Ali Ahmad. It used to depend on a service that was never delivered and a model at a path that doesn't exist on this machine — both were fixed to import the website's own live models directly, so the extension and the website can never disagree with each other.

Q: What does the badge on a post actually check?

The caption, through the same misinformation model plus a rule-based red-flag layer, and the poster's account through the same fake-account model.

Q: Does it work on video/reel audio?

Not yet — that needs the Whisper transcriber module, which isn't integrated. Without it, a reel is honestly labelled 'not checked', never assumed safe.

Q: Does it work on TikTok?

No — it's Instagram-only by design; the extension's page-reading code is written specifically for Instagram's page structure.

Admin Console

Q: Why does this exist?

Our scope document defines an Admin role responsible for user management, reviewing score disputes within 48 hours, and model performance monitoring — this is that role, built out as a real, working console.

Q: How does a user actually dispute a score?

From the Results page, 'Dispute this score' with a reason and evidence link. It appears in the admin queue with a live 48-hour countdown. An admin reviews it and either upholds it with a corrected score, or rejects it with an explanation — either way the user sees the decision on their Dashboard.

Q: Can an admin just give themselves admin access?

No — the very first admin account can only be created from the command line with a script, never through the website. After that, only an existing admin can promote someone else.

Q: What stops an admin abusing this?

Several server-enforced rules: an admin cannot suspend or demote themselves, cannot decide a dispute they personally filed, and every action is written to an audit log with who/when/why.

5. The Whole Project, in Phases

If asked to walk through the system end to end, use these six phases in order — this is the actual order data flows through the app.

Phase 1 — Data comes in

User types a username on the Landing page → frontend calls POST /analyze-live → backend fetches the public profile, recent posts, and comments from RapidAPI.

backend/data_ingestion.py	fetch_instagram_profile(), fetch_engagement_data(), fetch_post_comments() — the three RapidAPI calls.
backend/main.py	The /analyze-live endpoint that calls all three, in parallel where possible.

Phase 2 — Each module scores independently

The same raw data is handed to four independent checks, each of which knows nothing about the others.

backend/fake_follower.py	The live Random Forest — Account Authenticity.
backend/engagement_analyzer.py	Engagement rate + comment authenticity.
backend/credential_extractor.py	Bio credential-claim detection.
backend/misinfo_assessment.py	Wraps the AI classifier with the red-flag cross-check, on bio + every caption.
backend/misinfo_classifier/api/model_loader.py	Loads the actual XLM-RoBERTa model and runs classify_text().
backend/red_flags.py	The hand-written scam-pattern rules, shared by the website and the extension.

Phase 3 — The four scores combine into one

Whichever modules actually applied get their weights re-normalized to 100%, the kill-switch is checked, and one 0-100 score plus a verdict comes out.

backend/trust_score.py	calculate_trust_score() and verdict_for() — the entire scoring formula lives in this one file.
------------------------	--

Phase 4 — Saved and shown to the user

The full result is saved to the database and sent back to the browser, which renders the Results page.

backend/database.py	SQLite tables: users, scan_history, appeals, audit_log.
frontend/src/pages/Results.jsx	The score ring, one-line summary, per-module breakdown, and dispute button.
frontend/src/api/trustlens.js	The only file that talks to the backend from the frontend.

Phase 5 — Around the core scan: accounts and history

Login/signup, personal scan history, and comparing two profiles side by side.

<code>backend/auth.py</code>	Signup, login, email verification, JWT tokens.
<code>frontend/src/context/AuthContext.jsx</code>	Holds the logged-in user across the whole app.
<code>frontend/src/pages/Dashboard.jsx</code>	Scan history + dispute status.
<code>frontend/src/pages/Comparison.jsx</code>	Two live scans side by side.

Phase 6 — The feedback loop

If a user disputes a score, an admin reviews it, and the outcome is recorded as labelled data for future retraining.

<code>backend/admin.py</code>	Every /admin endpoint — the whole admin API.
<code>frontend/src/components/DisputePanel.jsx</code>	The 'Dispute this score' form.
<code>frontend/src/pages/admin/</code>	9 admin pages: Overview, Appeals, Users, System, Audit, etc.

6. If Asked About Something Not Built

Do not bring these up yourself. If the panel asks directly, here is the one honest line for each — short, calm, no over-explaining.

If asked about...	Say exactly this
Deepfake Scanner	"It was built and briefly wired in, but removed on purpose before this review because it wasn't reviewed as a team and I want to rebuild it properly rather than ship something half-checked. It does not exist in the repo right now — that was a deliberate choice, not an oversight."
Visual Analytics	"Not started yet — the supervisor's guidance was to fully finish fewer modules rather than half-finish all twelve, so this one was deprioritised."
TikTok support	"Instagram-only right now, by design — the scope was to go deep on one platform first."
Video-to-text / Whisper transcriber	"Built once already by a teammate, and an updated version is coming — not integrated yet, so reel audio is honestly reported as 'not checked' rather than assumed safe."
PDF/CSV export, radar chart on Comparison	"Scoped for later — the core comparison already works live with real data, these are presentation extras on top of it."
External credential verification (LinkedIn/registries)	"Attempted, but the required Google API needed a live billing account, which was judged too risky this close to submission — deliberately deferred, documented as a known limitation, not hidden."

General rule for the whole day

Every honest 'not yet' above is followed by a reason — you were never lazy, you made a call under time pressure and can defend it. That is a strength in a viva, not a weakness. Panels notice confident honesty far more than they notice a missing feature.

End of Day 2. Tell Claude about the UI whenever you're ready to continue.